

HAProxy 기초 개념문제 30개

정답 + 자세한 해설 포함 (PDF)

생성일: 2026-01-16

활용 방법

- 1) 먼저 정답을 가리고(스크롤/화면 가림) 문제를 풀어보세요.
- 2) 해설을 읽을 때는 "왜 다른 선택지가 틀렸는지"까지 같이 확인하세요.
- 3) 헛갈린다면, 예시 설정 스니펫을 직접 손으로 타이핑해보는 게 가장 빠릅니다.
- 4) 운영 환경에서는 로그/헬스체크/타임아웃/리로드(무중단) 4가지를 특히 단단히 잡으세요.

팁) HAProxy는 "설정 파일을 읽어서 동작하는" 도구라서, 문법/구조를 이해하면 실력이 빠르게 늘어요.

- frontend: 클라이언트 접속 창구
- backend: 실제 서버 풀 + 로드밸런싱/헬스체크 정책
- ACL: 조건에 이름 붙이기(규칙 재사용)

핵심 설정 키워드(한 장 요약)

개념	설정 예시(발췌)
frontend/backend	frontend fe bind :80 default_backend be backend be server s1 10.0.0.10:80 check
HTTP/TCP 모드	mode http # L7(Host/Path/헤더) mode tcp # L4(연결 단위)
ACL 라우팅	acl is_api path_beg /api use_backend be_api if is_api
헬스체크	option httpchk GET /health server s1 10.0.0.10:80 check rise 3 fall 2
타임아웃	timeout connect 5s timeout client 30s timeout server 30s
Stats/로그	stats enable stats uri /stats option httplog

HAProxy의 대표적인 역할로 가장 알맞은 것은?

정답

B

- A. 데이터베이스 백업/복구 도구
- B. L4/L7 로드밸런서 및 리버스 프록시
- C. DNS 서버(도메인 → IP 변환)
- D. 패킷 캡처/분석 도구

```
frontend fe_http
  bind :80
  default_backend be_app
  backend be_app
server s1 10.0.0.10:80 check
```

해설

HAProxy는 클라이언트 트래픽을 "앞에서 받아서(Proxy)" 여러 서버로 "분산(Load Balancing)"하는 도구입니다.

1) L4(TCP) 레벨: 연결 단위로 분산(예: TCP 서비스, DB 프록시)

2) L7(HTTP) 레벨: Host/Path/헤더/쿠키를 보고 라우팅(예: 웹)

운영에서 자주 쓰는 기능은 헬스체크, TLS 종료(HTTPS), 세션 고정(Sticky), 로그/통계(Stats)입니다.

Host/Path 같은 HTTP 정보 기반으로 라우팅하려면 어떤 설정이 필요할까?

정답

B

- A. mode tcp
- B. mode http
- C. option tcplog
- D. maxconn

```
frontend fe
  mode http
  acl is_api path_beg /api
  use_backend be_api if is_api
```

해설

Host/Path/헤더/쿠키는 HTTP(L7) 정보입니다. HAProxy는 mode http에서 HTTP 파서를 사용하므로, path_beg, hdr(host), method 같은 ACL 조건을 사용할 수 있습니다. mode tcp는 "바이트 스트림"만 다루기 때문에 HTTP 헤더 기반 라우팅을 기본적으로 할 수 없습니다.

HAProxy 설정에서 frontend의 역할로 가장 알맞은 설명은?

정답

C

- A. 실제 서버 목록을 나열하는 곳
- B. 인증서 파일을 보관하는 폴더
- C. 클라이언트 연결을 받는 "접수 창구"(bind/규칙/라우팅)
- D. 로그 파일을 회전(rotate)하는 기능

```
frontend fe_web
  bind :80
  default_backend be_web
```

해설

frontend는 "클라이언트가 접속하는 입구"입니다.
여기서 어떤 IP:Port로 받을지(bind), 어떤 조건이면 어느 backend로 보낼지(ACL/use_backend)를 정의합니다.
실제 서버 풀(server 라인)은 backend에 두는 것이 기본 구조입니다.

bind 지시어(directive)의 주된 목적은?

정답

B

- A. 로그 레벨을 설정한다
- B. HAProxy가 듣고 있을 IP/Port(리스닝 소켓)를 연다
- C. 서버에 접속할 사용자 계정을 지정한다
- D. 백엔드 서버의 가중치를 자동 계산한다

```
frontend fe_https
  bind :443 ssl crt
  /etc/haproxy/certs/site.pem
```

해설

bind는 "클라이언트가 들어오는 포트"를 열어주는 설정입니다.

예) bind :80, bind 0.0.0.0:443 ssl crt ...

보통 frontend 또는 listen 섹션에 위치합니다(backend는 서버로 "나가는" 쪽이라 bind가 핵심이 아닙니다).

다음 ACL 설정이 의미하는 바는? `acl is_api path_beg /api`

정답

A

- A. /api로 시작하면 is_api라는 "조건 이름"을 참으로 만든다
- B. /api 경로를 강제로 차단한다
- C. /api로 리다이렉트한다
- D. API 서버를 자동으로 추가한다

```
acl is_api path_beg /api
```

해설

ACL은 "조건에 이름을 붙이는 문법"입니다.
is_api라는 이름의 조건을 만들고, 요청 경로가 /api로 시작하면 참(True)이 됩니다.
이 조건을 if/unless에 붙여서 라우팅, 차단, 헤더 추가 같은 동작을 구성합니다.

use_backend be_api if is_api 의 동작으로 맞는 것은?

정답

B

- A. 항상 be_api로 보낸다
- B. is_api 조건이 참일 때만 be_api로 보낸다
- C. be_api 서버들을 모두 다운(OFF) 처리한다
- D. be_api를 생성만 하고 사용하지 않는다

```
use_backend be_api if is_api
```

해설

use_backend는 "조건에 따라 어느 backend로 보낼지"를 결정합니다.
is_api가 참인 요청만 be_api로 라우팅하고, 그렇지 않으면 아래 규칙 또는 default_backend로 넘어갑니다.
여러 use_backend가 있다면 위에서부터 순서대로 검사하므로, 규칙 순서가 중요합니다.

default_backend의 역할은 무엇일까?

정답

A

- A. 규칙에 매칭되지 않은 트래픽의 "기본 목적지"를 지정한다
- B. HAProxy의 기본 포트를 80으로 바꾼다
- C. 기본 로그 파일 경로를 지정한다
- D. 기본 인증서를 자동 선택한다

```
default_backend be_web
```

해설

default_backend는 "어떤 use_backend 조건도 만족하지 않을 때" 보내는 기본 backend입니다.
라벨을 붙이지 않은 일반 트래픽(예: /)을 기본 서비스로 보내는 용도로 자주 사용됩니다.
default_backend가 없고 라우팅 규칙도 없으면, 요청이 갈 곳이 없어 503을 만들 수 있습니다.

헬스체크(health check)의 핵심 목적은?

정답

B

- A. 서버 성능을 벤치마크한다
- B. "정상 서버만" 로드밸런싱 대상에 남기기 위해 주기적으로 상태를 확인한다
- C. 패킷을 암호화한다
- D. 도메인 이름을 IP로 변환한다

```
option httpchk GET /health  
server s1 10.0.0.10:80 check
```

해설

헬스체크는 "서버가 살아있는지/정상 응답하는지"를 주기적으로 확인해, 문제가 있는 서버를 자동으로 풀에서 빼고(Down), 회복되면 다시 넣는(Up) 기능입니다. 이 기능이 없으면 장애 서버로도 계속 트래픽이 가서 사용자 오류가 커집니다.

rise/fall 설정의 의미로 맞는 것은?

정답

B

- A. rise는 실패 횟수, fall은 성공 횟수
- B. rise는 연속 성공 횟수, fall은 연속 실패 횟수(Up/Down 판정)
- C. rise/fall은 로드밸런싱 알고리즘 이름
- D. rise/fall은 로그 포맷 설정

```
server s1 10.0.0.10:80 check rise 2 fall 3
```

해설

rise/fall은 헬스체크 결과가 "잠깐 흔들리는 것(플랩)"을 완화하기 위한 장치입니다.

- fall 3: 3번 연속 실패해야 Down
- rise 2: 2번 연속 성공해야 Up

즉, 일시적인 실패로 서버를 바로 제외하지 않도록(또는 반대로 너무 빨리 복귀시키지 않도록) 안정성을 높입니다.

roundrobin 알고리즘에 대한 설명으로 맞는 것은?

정답

B

- A. 서버의 현재 연결 수가 가장 적은 곳에 보낸다
- B. 서버를 순서대로 돌아가며(라운드로빈) 분산한다
- C. 항상 첫 번째 서버로 보낸다
- D. 클라이언트 IP가 같으면 같은 서버로 고정한다

balance roundrobin

해설

roundrobin은 가장 기본적인 분산 방식으로, 새 요청/연결을 서버 목록에 순서대로 배정합니다.
요청 처리 시간이 비슷하고 서버 성능이 비슷한 환경에서는 단순하고 효과적입니다.
단, "연결이 오래 유지되는 트래픽"에서는 부하 편차가 생길 수 있어 leastconn이 더 적합할 때가 있습니다.

leastconn 알고리즘이 특히 유리한 상황은?

정답

B

- A. 모든 요청이 짧고 동일한 처리 시간일 때
- B. 연결/요청 처리 시간이 제각각이고 오래 유지되는 연결이 많을 때
- C. 서버가 1대뿐일 때
- D. DNS가 불안정할 때

balance leastconn

해설

leastconn은 "현재 활성 연결 수가 가장 적은 서버"를 우선 선택합니다.
웹소켓, 긴 다운로드, 장시간 세션처럼 "연결 유지 시간이 길거나 들쭉날쭉한" 트래픽에서 부하 균형에 유리합니다.
단, keep-alive 방식/애플리케이션 특성에 따라 체감이 달라질 수 있어 실측이 중요합니다.

server 라인에 weight 2를 주면 일반적으로 어떤 의미일까?

정답

A

- A. 해당 서버로 트래픽을 2배 더 보내려는 "비율" 힌트
- B. 서버 CPU를 2개로 늘린다
- C. 서버의 포트를 2개 연다
- D. 헬스체크를 2초마다 한다

```
server s1 10.0.0.10:80 check weight 2
```

해설

weight는 서버별 "트래픽 분배 비율"을 조정합니다.
예를 들어 서버 A weight 2, 서버 B weight 1이면, 새 트래픽이 대략 2:1 비율로 배정됩니다.
서버 성능이 서로 다르거나(스펙/용량) 점진적 마이그레이션을 할 때 활용합니다.

HTTP 기반 Sticky Session(세션 고정)을 대표적으로 구현하는 방법은?

정답

A

- A. 쿠키(cookie)를 이용해 같은 클라이언트를 같은 서버로 보내기
- B. DNS TTL을 줄이기
- C. syslog를 끄기
- D. timeout connect를 0으로 만들기

```
backend be_web
  cookie SRV insert indirect nocache
server s1 10.0.0.10:80 cookie s1 check
```

해설

상태를 서버 로컬에 들고 있는(세션 상태) 애플리케이션은 같은 사용자가 계속 같은 서버로 가야 안정적인 경우가 있습니다. 그 대표 방법이 "쿠키 기반 Sticky"입니다. HAProxy가 서버 식별 쿠키를 넣어주고, 브라우저가 이후 요청에 쿠키를 포함하면 같은 서버로 라우팅합니다. 가능하면 애플리케이션을 무상태(Stateless)로 만드는 것이 장기적으로 더 좋습니다.

balance source(또는 sticky on src)의 특징으로 맞는 것은?

정답

B

- A. HTTP 쿠키가 있어야만 동작한다
- B. 클라이언트 IP 해시 기반으로 같은 IP는 같은 서버로 갈 확률이 높다
- C. 서버 응답 본문을 분석해 라우팅한다
- D. TLS 인증서를 자동 발급한다

balance source

해설

source 기반 분산은 "클라이언트 IP"를 해시해 서버를 선택합니다.

장점: 설정이 단순하고 TCP 모드에서도 적용 가능

주의: NAT 환경에서는 많은 사용자가 같은 IP로 보여 특정 서버에 쏠릴 수 있고, 사용자 IP가 바뀌면 고정기 깨질 수 있습니다.

option forwardfor가 하는 일로 맞는 것은?

정답

B

- A. 요청을 다른 포트로 포워딩한다
- B. X-Forwarded-For 헤더에 원본 클라이언트 IP를 추가한다
- C. 방화벽 정책을 자동 생성한다
- D. 서버 응답을 캐시한다

option forwardfor

해설

리버스 프록시 뒤에 있으면 백엔드 서버 로그에 "프록시 IP만" 남는 문제가 생깁니다.
option forwardfor는 HTTP 요청에 X-Forwarded-For 헤더를 추가해 원본 클라이언트 IP를 전달합니다.
중요: 애플리케이션은 이 헤더를 "신뢰할 수 있는 프록시에서만" 받아들이도록 설정해야 합니다(스푸핑 방지).

timeout connect/client/server의 의미를 올바르게 설명한 것은?

정답

A

- A. connect=서버 연결 시도 시간, client=클라이언트 쪽 유희, server=서버 쪽 유희
- B. connect=로그 저장 시간, client=쿠키 유효 시간, server=인증서 유효 시간
- C. 모두 같은 의미로 중복 설정이다
- D. connect는 HTTP 헤더 타임아웃이다

```
timeout connect 5s
timeout client 30s
timeout server 30s
```

해설

timeout은 운영 안정성의 핵심입니다.

- timeout connect: 백엔드 서버로 TCP 연결을 맺는 데 허용하는 최대 시간(느린/죽은 서버를 빨리 감지)
 - timeout client: 클라이언트 방향 트래픽이 없을 때 끊기까지의 유희 시간
 - timeout server: 서버 방향 트래픽이 없을 때 끊기까지의 유희 시간

서비스 특성(업로드/다운로드, 장시간 요청)에 맞게 튜닝해야 "끊김"과 "좀비 연결"을 둘 다 피할 수 있습니다.

maxconn 설정은 무엇을 제어하는가?

정답

B

- A. 최대 응답 본문 크기
- B. HAProxy가 동시에 처리(수락)할 수 있는 최대 연결 수
- C. 최대 서버 대수
- D. 최대 로그 파일 크기

```
global
maxconn 4000
frontend fe
maxconn 2000
```

해설

maxconn은 과부하를 막는 "안전장치"입니다.

동시에 처리할 수 있는 연결 수를 제한해 HAProxy 자체가 메모리/CPU로 죽는 상황을 줄입니다.
global의 maxconn은 전체 한도, frontend의 maxconn은 특정 리스너 한도를 의미할 수 있습니다.
한도를 넘으면 대기열(큐) 또는 거절(설정/상황에 따라)로 이어질 수 있습니다.

HAProxy Stats 페이지의 주된 용도로 가장 알맞은 것은?

정답

B

- A. 커널 파라미터를 수정한다
- B. 프론트/백엔드/서버 상태와 트래픽 지표를 실시간으로 확인한다
- C. SSL 인증서를 자동으로 갱신한다
- D. 방화벽을 우회한다

```
listen stats
  bind :8404
  stats enable
  stats uri /stats
  stats auth admin:pass
```

해설

Stats는 HAProxy의 "상태/지표 대시보드"입니다.
서버 Up/Down, 현재 연결 수(scur), 큐(qcur), 응답 코드, 에러 등을 한눈에 볼 수 있어 장애 분석에 매우 유용합니다.
보안상 외부 노출은 피하고, 내부망 + 인증(또는 접근제어)로 보호하는 것이 좋습니다.

option httplog와 option tcplog의 차이로 맞는 것은?

정답

A

- A. httplog는 HTTP 요청 정보(메서드/경로/상태코드)를 더 잘 남긴다
- B. tcplog는 HTTP 헤더를 파싱해 URL을 기록한다
- C. 둘 다 동일하며 아무거나 써도 된다
- D. tcplog는 TLS 인증서 내용을 로그로 남긴다

```
option httplog
log /dev/log local0
```

해설

로그 포맷은 모드에 맞게 선택해야 합니다.

- option httplog: HTTP 기반 로그(메서드/URI/상태코드/응답시간 등)가 풍부
 - option tcplog: TCP 연결 단위 로그(연결/종료/바이트 등)

mode tcp에서 httplog를 쓰면 기대한 HTTP 정보가 없고, 반대로 mode http에서 tcplog를 쓰면 디버깅이 불편해질 수 있습니다.

“SSL/TLS Termination”에 대한 설명으로 맞는 것은?

정답

A

- A. HAProxy가 HTTPS를 받아서 복호화한 뒤 백엔드에는 HTTP로 전달하는 구성
- B. 백엔드 서버가 모두 같은 인증서를 자동으로 생성하는 기능
- C. 네트워크 패킷을 캡처해서 암호화하는 기능
- D. DNS 레코드를 자동 변경하는 기능

```
bind :443 ssl crt  
/etc/haproxy/certs/site.pem
```

해설

TLS 종료(termination)는 HTTPS 핸드셰이크/암복호화를 HAProxy가 담당하는 방식입니다.
장점: 인증서 관리/갱신을 중앙에서 처리, L7 라우팅/헤더 조작/리다이렉트 같은 기능이 쉬워짐
주의: 백엔드까지 암호화가 필요하면 재암호화(SSL re-encrypt) 또는 백엔드도 HTTPS로 구성합니다.

HTTP(80)로 들어온 요청을 HTTPS로 강제하려면 어떤 방식이 가장 일반적인가?

정답

B

- A. mode tcp에서 redirect 사용
- B. http-request redirect scheme https code 301
- C. timeout server를 1ms로 줄인다
- D. balance source로 변경한다

```
frontend fe_http
  bind :80
  mode http
  http-request redirect scheme https code
    301
```

해설

리다이렉트는 "HTTP 응답(301/302)"을 보내는 동작이라 mode http에서 수행합니다.
일반적으로 80 포트 frontend에서 HTTPS로 리다이렉트하고, 실제 서비스는 443 포트 frontend에서 처리합니다.
주의: 운영에서는 HSTS까지 켜기 전에 HTTPS가 완전하게 동작하는지(리소스/서브도메인 포함) 먼저 확인하세요.

HSTS(Strict-Transport-Security) 헤더를 추가하려면 주로 어떤 지시어를 쓸까?

정답

A

- A. http-response set-header
- B. tcp-request content
- C. option tcplog
- D. maxconn

```
http-response set-header Strict-Transport-  
Security "max-age=31536000;  
includeSubDomains"
```

해설

HSTS는 "브라우저에게 앞으로는 HTTPS만 사용하라"고 알려주는 응답 헤더입니다. 따라서 서버(HAProxy)가 클라이언트에게 보내는 "응답(response)"에 헤더를 추가해야 하므로 http-response set-header를 사용합니다.

주의: HSTS를 켜면 브라우저가 HTTP 접속을 막을 수 있어, HTTPS 구성이 완벽할 때만 적용하세요.

PROXY protocol을 사용하는 가장 대표적인 이유는?

정답

B

- A. HTTP 요청 본문을 압축하기 위해
- B. TCP 프록시에서 백엔드로 "원본 클라이언트 IP/Port" 같은 정보를 전달하기 위해
- C. 인증서를 자동 갱신하기 위해
- D. 로드밸런싱 알고리즘을 빠르게 하기 위해

```
server app1 10.0.0.10:443 send-proxy
```

해설

X-Forwarded-For는 HTTP 헤더라서 mode http에서만 자연스럽게 다룹니다.
TCP 모드(또는 L4 프록시)에서 원본 클라이언트 정보를 백엔드에 전달하려면 PROXY protocol이 대표적입니다.
주의: 백엔드도 PROXY protocol을 "받아들일 준비(accept-proxy)"가 되어 있어야 하며, 양쪽 설정이 맞아야 합니다.

server ... check inter 2s 에서 inter 2s는 무엇을 의미할까?

정답

A

- A. 헬스체크 간격(주기)을 2초로 설정
- B. 서버 응답 시간 2초를 강제
- C. 최대 연결 수를 2개로 제한
- D. 쿠키 유효 시간을 2초로 설정

```
server s1 10.0.0.10:80 check inter 2s rise  
3 fall 2
```

해설

inter는 헬스체크 "주기(간격)"입니다.

- inter 2s: 2초마다 체크

너무 짧으면 체크 트래픽이 많아지고, 너무 길면 장애 감지가 느려집니다.

rise/fall과 함께 조정해 "빠른 감지 + 안정적인 판정"을 균형 있게 맞추는 게 좋습니다.

stick-table을 이용한 레이트 리밋(rate limit)의 개념으로 맞는 것은?

정답

B

- A. 서버 CPU를 자동으로 낮춘다
- B. 클라이언트(예: IP)별로 요청/연결 카운터를 저장해 과도한 요청을 제한한다
- C. 인증서를 자동 발급한다
- D. DNS 캐시를 비운다

```
stick-table type ip size 100k expire 10m
      store http_req_rate(10s)
http-request deny if { src_http_req_rate
      gt 100 }
```

해설

stick-table은 "키(예: IP, 세션ID) → 상태/카운터"를 저장하는 작은 인메모리 테이블입니다. 이를 이용하면 특정 IP가 10초 동안 너무 많은 요청을 보내는지(http_req_rate) 같은 지표를 계산해, 임계치를 넘으면 차단/지연 같은 방어를 할 수 있습니다. 기초 단계에서는 '원리(키별 카운터)'만 이해해도 운영에서 큰 도움이 됩니다.

HAProxy가 클라이언트에게 503을 반환하는 대표적인 상황은?

정답

A

- A. 백엔드 서버가 모두 Down이라서 보낼 곳이 없을 때
- B. 항상 애플리케이션 코드 버그(500) 때문이다
- C. DNS가 느릴 때만 발생한다
- D. 인증서 만료와 무조건 동일하다

```
stats uri /stats # 상태 확인
```

해설

503은 "HAProxy가 요청을 처리했지만 백엔드로 전달하지 못했다"는 신호인 경우가 많습니다.

대표 원인

- 1) 백엔드 서버가 모두 Down(헬스체크 실패)
- 2) maxconn 제한으로 대기/거절
- 3) 라우팅 규칙/백엔드 누락(갈 곳이 없음)

장애 시에는 Stats/로그에서 서버 상태와 에러 카운터를 먼저 확인하는 것이 빠릅니다.

option redispatch의 의미로 가장 알맞은 것은?

정답

A

- A. 선택된 서버가 불가하면 다른 서버로 재선택(재분배)할 수 있게 한다
- B. 로그를 다른 파일로 이동(dispatch)한다
- C. SSL 인증서를 재발급한다
- D. 클라이언트 IP를 숨긴다

option redispatch

해설

로드밸런싱 과정에서 어떤 이유로 특정 서버가 선택되었는데(예: 세션 고정),
그 서버가 막상 연결 불가/Down이면 실패가 날 수 있습니다.
option redispatch는 이런 상황에서 "다른 서버로 재시도"할 수 있게 도와 성공률을 높입니다.
단, 엄격한 sticky 정책이 필요하면 영향이 있을 수 있어 의도적으로 사용해야 합니다.

무중단에 가깝게 설정 변경을 적용하는 “그레이스풀 리로드” 개념으로 맞는 것은?

정답

B

- A. 프로세스를 완전히 종료하고 다시 켜다(기존 연결 모두 끊김)
- B. 새 프로세스를 띄우고 기존 프로세스는 기존 연결을 처리한 뒤 종료한다
- C. 서버를 포맷한다
- D. DNS TTL을 0으로 만든다

```
# (리눅스 예시) 기존 PID에 soft-stop 전달
haproxy -f /etc/haproxy/haproxy.cfg -p
/run/haproxy.pid -sf $(cat
/run/haproxy.pid)
```

해설

HAProxy는 설정 변경 시 “새 프로세스(새 설정)”를 띄우고,
기존 프로세스는 “기존 연결”을 처리한 다음 자연스럽게 내려가는 방식으로 다운타임을 줄일 수 있습니다.
운영 포인트는 두 가지입니다.

- 1) 리로드 전에 반드시 설정 검증(-c)
- 2) 리로드 절차(시그널/피드파일/서비스 매니저)가 표준화되어 있어야 한다

haproxy -c -f haproxy.cfg 명령의 목적은?

정답

A

- A. 설정 파일 문법/유효성을 검사하고 종료한다(실행 X)
- B. HAProxy를 백그라운드로 실행한다
- C. 인증서를 갱신한다
- D. 모든 백엔드 서버를 Down 처리한다

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

해설

-c 옵션은 "체크(check)" 모드입니다.

운영에서 리로드 직전에 이 검증을 자동화해두면, 문법 실수로 서비스가 죽는 사고를 크게 줄일 수 있습니다.
검증이 통과하면 그 다음에만 리로드/재시작을 수행하는 흐름이 베스트 프랙티스입니다.

http-request set-header 와 http-response set-header
의 차이로 맞는 것은?

정답

B

- A. 둘 다 응답에만 적용된다
- B. request는 백엔드로 가는 요청을, response는 클라이언트로 가는 응답을 수정한다
- C. 둘 다 TCP 모드에서만 동작한다
- D. set-header는 쿠키를 자동으로 만든다

```
http-request set-header X-Forwarded-Proto
                    https if { ssl_fc }
http-response set-header X-Frame-Options
                    DENY
```

해설

방향(흐름)을 정확히 구분하면 설정이 쉬워집니다.

- http-request set-header: "클라이언트 → HAProxy → 백엔드"로 전달되는 요청을 수정(예: X-Forwarded-Proto 추가)
 - http-response set-header: "백엔드 → HAProxy → 클라이언트"로 전달되는 응답을 수정(예: 보안 헤더 추가)
- 둘 다 mode http에서 의미가 크며, 운영에서는 보안/관측 헤더를 일괄로 넣는 데 자주 사용합니다.